Rapport de projet : Jeu PieceOut

Plan

1.	CE QUI A ÉTÉ TRAITÉ
2.	ASPECTS LES PLUS SIGNIFICATIFS
3.	QUELQUES PROBLÈMES CONNUS
4.	ETATS EXTENSIONS
5.	DIAGRAMMES UML

Rapport du Projet

Auteurs

- Abdoulbasti MUKAILAALH SHITTU 22024114
- Fatoumata NDOUR 22321513

Année Universitaire

2024/2025

Master 1 Informatique Mathématiques– Université de Paris Cité

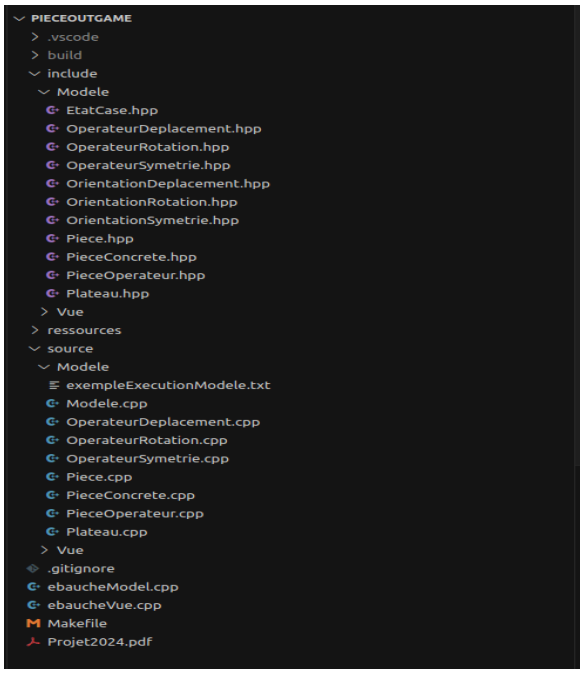
Introduction

Ce rapport présente le projetPièceOut, un jeu inspiré de casse-têtes où l'objectif est de résoudre des puzzles en déplaçant et en manipulant des pièces selon des règles spécifiques. Ce projet avait pour but de démontrer la maîtrise des concepts enseignés au cours du semestre, notamment :

- La construction, destruction, copie et utilisation des objets en C++.
- Les concepts tels que l'héritage, la généricité, les exceptions, et les clients de conception (exemple :Décorateur et Visiteur).
- La conception et la structuration d'un code clair et maintenable, en séparant le modèle et la vue.

L'évaluation inclut également une démonstration du jeu, un examen du code et de sa conception via des diagrammes UML, ainsi qu'une discussion sur les choix de conception et d'implémentation.

1. CE QUI A ÉTÉ TRAITÉ



1. 1 Architecture Modèle et Vue

La partie Modèle : regroupe toute la logique du jeu PieceOut, y compris les règles, le comportement des pièces, et les calculs pour la détection des gains.

La partie Vue : dédiée à la gestion de l'interface utilisateur et de l'affichage graphique. Cette séparation a été facilitée par l'utilisation de fichiers d'en-tête .hpp pour les définitions et de fichiers sources .cpp pour les implémentations

1. 2 Makefile pour la compilation

Un Makefile a été conçu pour simplifier la compilation et la gestion de PieceOut. Il inclut des cibles pour : Compiler les fichiers individuellement, Générer un exécutable, Nettoyer les fichiers temporaires ou les binaires.

Makefile

```
CPP=g++ --std=c++11 -Wall -Wextra -Iinclude
LSFML_OPTIONS=-lsfml-system -lsfml-graphics -lsfml-window -lsfml-audio
SOURCE_MODELE_DIR=source/Modele
INCLUDE_MODELE_DIR=include/Modele
SOURCE_VUE_DIR=source/Vue
INCLUDE_VUE_DIR=include/Vue
BD_DIR=build

# La Liste des fichiers objets pour le modele
#Ajouter ici d'autres objet .o
OBJ_MODELE_LIST =   $(BD_DIR)/Modele.o  $(BD_DIR)/Piece.o $(BD_DIR)/PieceConcrete.o  $(BD_DIR)/PieceOperateur.o $(BD_DIR)/OperateurDeplacement.o \
                    $(BD_DIR)/OperateurRotation.o  $(BD_DIR)/OperateurSymetrie.o $(BD_DIR)/Plateau.o

# La Liste des fichiers objets pour la vue
#Ajouter ici d'autres objet .o
OBJ_VUE_LIST = $(BD_DIR)/Vue.o $(BD_DIR)/VuePlateau.o $(BD_DIR)/Piece.o $(BD_DIR)/PieceConcrete.o  $(BD_DIR)/PieceOperateur.o $(BD_DIR)/OperateurDeplacement.o \
                    $(BD_DIR)/OperateurRotation.o  $(BD_DIR)/OperateurSymetrie.o  $(BD_DIR)/Plateau.o $(BD_DIR)/Decorer.o $(BD_DIR)/Puzzle1.o $(BD_DIR)/Puzzle2.o $(BD_DIR)/Puzzle3.o $(BD_DIR)/Puzzle4.o


#------------------------------------ compilation du modele ----------------------------------------#
all_modele : $(BD_DIR)/ProgramModele

#Compilation et excution
$(BD_DIR)/ProgramModele : $(OBJ_MODELE_LIST)      You, last month • refactor : factorisation du Makefile pour le mo…
    $(CPP) -o $(BD_DIR)/ProgramModele  $(OBJ_MODELE_LIST)
    ./$(BD_DIR)/ProgramModele

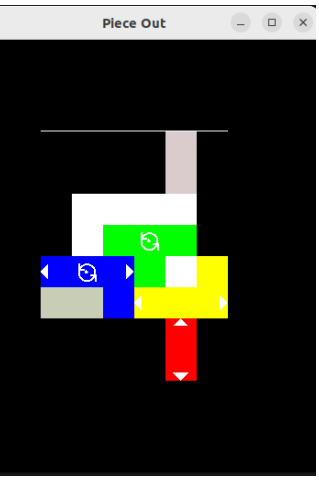
$(BD_DIR)/Modele.o : $(SOURCE_MODELE_DIR)/Modele.cpp
    mkdir -p build
    $(CPP) -c $(SOURCE_MODELE_DIR)/Modele.cpp -o $(BD_DIR)/Modele.o
```

1. 3 Décoration des pièces du jeu

Toutes les pièces du jeu sont entièrement décorables en fonction des décorations attribuées, qui varient en fonction de l'espace de jeu sélectionné par l'utilisateur.
Les types de décorations implémentées sont :

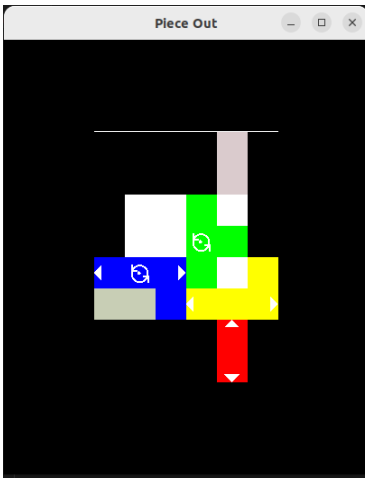
-Le Déplacement : Déplacement vers NORD, EST, SUD ou OUEST des pièces.

-La Symétrie : effets visuels reflétant la pièce horizontalement ou verticalement.
-La Rotation : rotation HORAIRE et ANTIHORAIRE des pièces sur l'espace de jeu.



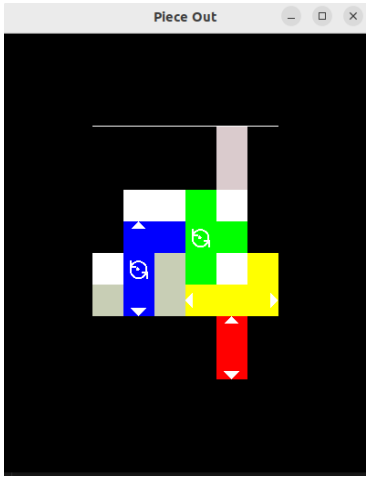
ANTIHORAIRE sur pièce Vert

---------->



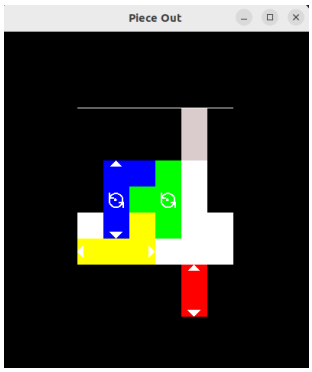
ANTIHORAIRE sur pièce Bleu

---------->



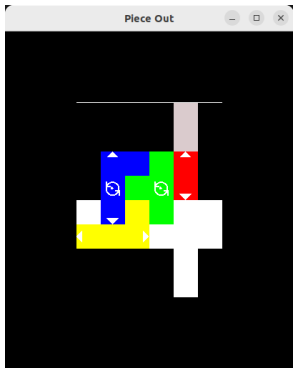
1. 4 Développement de l'espace de jeu espace de jeu et détection de gain

L'espace de jeu correspond à une grille qui sert à stocker les pièces du joueur et à exécuter les décorations associées, dans la mesure où cela est possible.
Le joueur peut interagir avec cet espace en appliquant des décorations spécifiques (déplacement, symétrie, rotation) aux pièces. Ces actions sont effectuées jusqu'à ce qu'une situation de gain soit détectée.
Une fois la condition de gain remplie, le jeu détecte automatiquement la victoire et met fin à la partie



Decorations

---------->



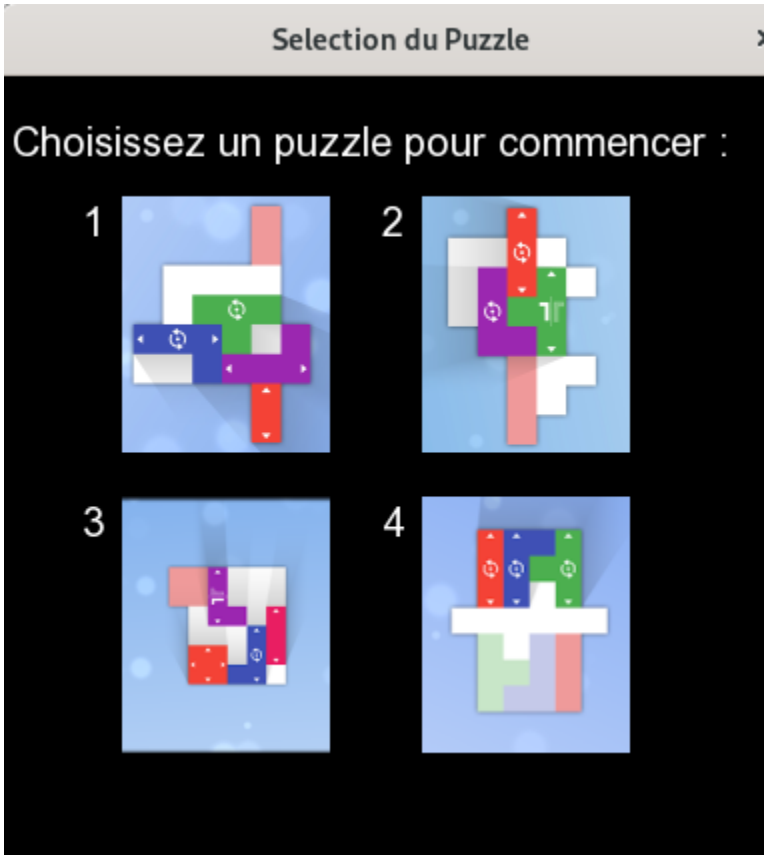
Decorations

---------->



1.5. Un environnement d’accueil graphique

L'environnement d'accueil du jeu présente une interface graphique simple et intuitive, où l'utilisateur est accueilli par un message d'instruction clair. En haut de la fenêtre, un texte explique à l'utilisateur : « Choisissez parmi les puzzles ci-dessous pour commencer une partie ». Cette instruction guide l'utilisateur dans le processus de sélection du puzzle, en l'invitant à cliquer sur l'une des quatre icônes de puzzle numérotées de 1 à 4. Ces icônes sont disposées de manière soignée et facilement accessibles. Chaque image représente un puzzle différent, et un simple clic sur l'une de ces images permet de démarrer le jeu et de commencer à jouer. Ce design simple et épuré permet une navigation fluide et une prise en main rapide du jeu, assurant ainsi une expérience utilisateur agréable dès l'écran d'accueil.



2. ASPECTS LES PLUS SIGNIFICATIFS

2.1 Utilisation des patrons de conception

- Le patron ***Decorator*** a été utilisé pour ajouter dynamiquement des **décorations** différentes aux pièces sans modifier leur structure initiale.
- Le patron ***Visitor*** a permis de parcourir les éléments du modèle de manière élégante pour effectuer des actions comme la détection de gain.

2.2 Séparation Modèle-Vue

La séparation entre le modèle et la vue garantit que la logique métier (règles du jeu) est indépendante du rendu graphique. Cela facilite la maintenance et l’évolution du projet.

2.3 Interface graphique

Mise en place de visualisation graphique du jeu Piece Out pour améliorer l'expérience utilisateur.

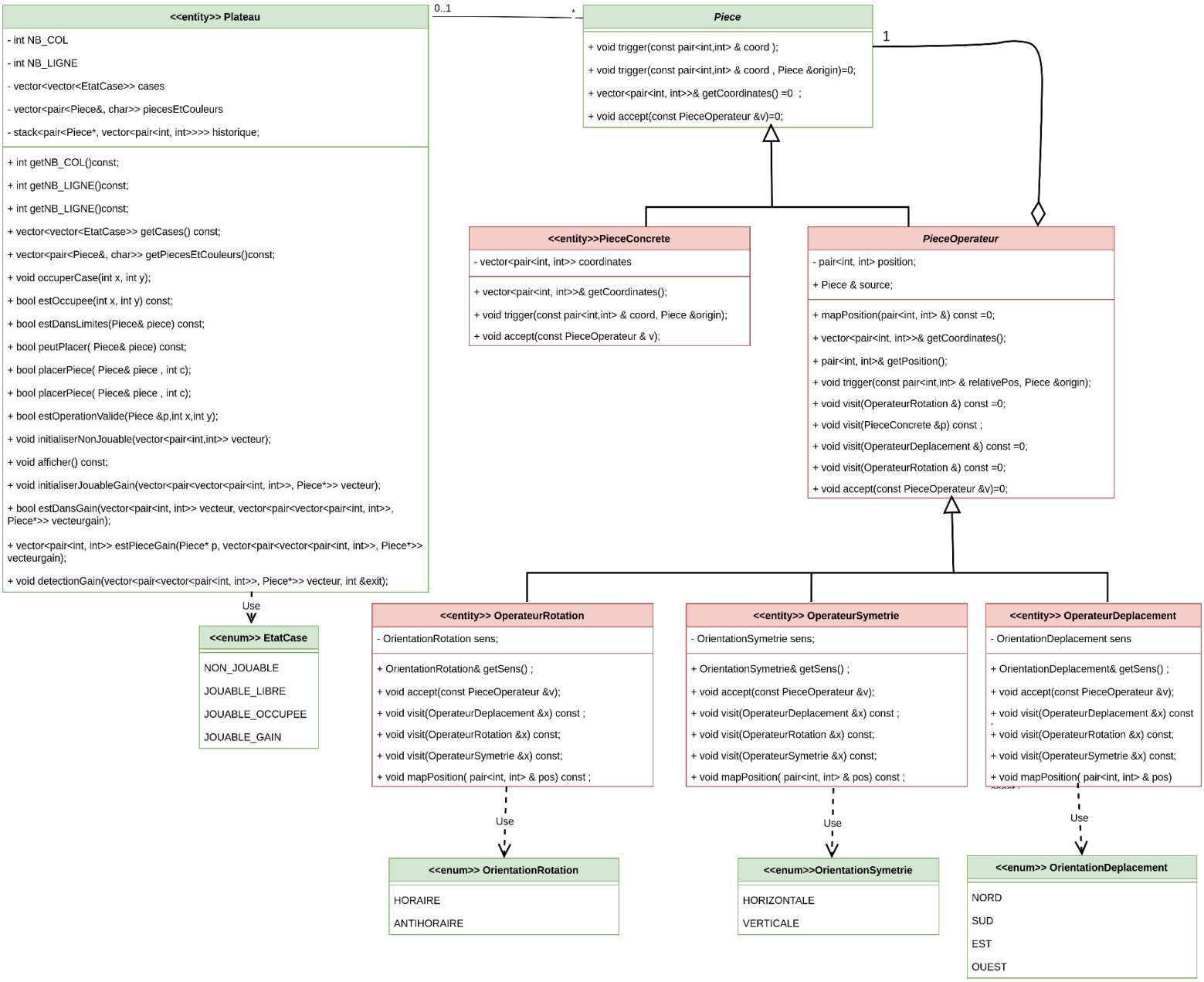
3. QUELQUES PROBLÈMES CONNUS

- Performances : Le rendu peut être lent pour des puzzles très complexes en raison d'une gestion sous-optimale des calculs de collision entre pièces.
- Documentation incomplète : Bien que le code soit globalement commenté, certaines parties complexes mériteraient des explications plus détaillées.

4. ETATS EXTENSIONS

- Retour arrière : Une fonctionnalité pour annuler les mouvements déjà effectués(UNDO), était en cours de développement.
- Mode multijoueur : La possibilité de jouer à deux en mode compétitif ou coopératif.
- Génération automatique de puzzles : Ajouter un algorithme génératif pour créer de nouveaux puzzles aléatoires.

5. DIAGRAMMES UML



CONCLUSION

Le projet **PieceOut** a permis de mettre en pratique les concepts avancés enseignés en **C++**, ainsi que la **conception** d'une solution avec **UML** et les **DESIGN PATTERN**. Bien que certaines extensions n'aient pas pu être implémentées, le jeu est fonctionnel, offre une expérience utilisateur satisfaisante, et démontre une **architecture** bien conçue. Ce travail constitue une base solide pour des améliorations futures.